PRÁCTICA GUIADA

PARTE 2

SHEILA CERQUEIRAS CASTRO

1°DAW

ÍNDICE

Actividad 6: Compilando y generando el Bytecode de la solución 4
Actividad 6: IDE IntelliJ 5
Actividad 7: 8
Actividad 8: 12
Actividad 9: 14

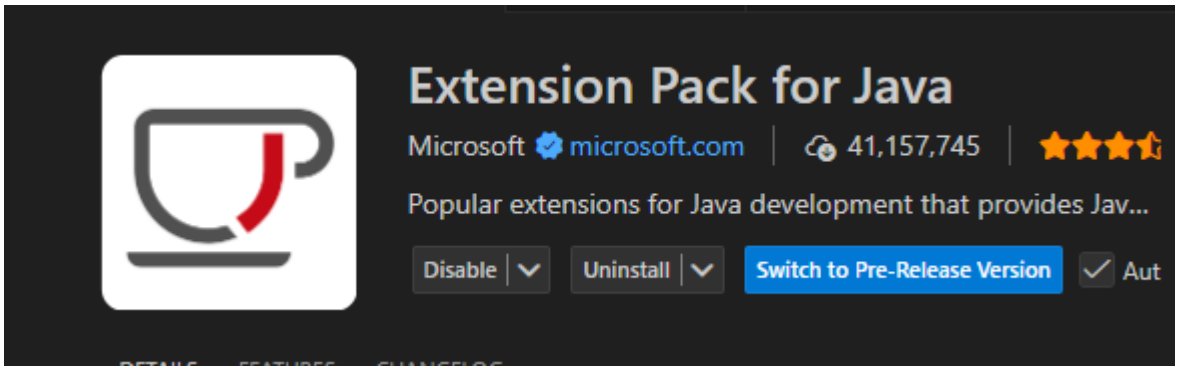
Actividad 5:
Configuración del IDE para otro lenguaje: Java Ahora vamos a ejecutar algunos proyectos en Java. Para ello hay que descargar y descomprimir el archivo adjunto “DepurarJava.zip” aquí encontraremos varias carpetas que iremos abriendo y viendo lo que tienen.

- **Abre la carpeta “Depurar1” y trata de ejecutar el fichero “Principal.java”. ¿Puedes ejecutarlo?**

No podemos y nos sale una notificación sobre pedirnos una extensión

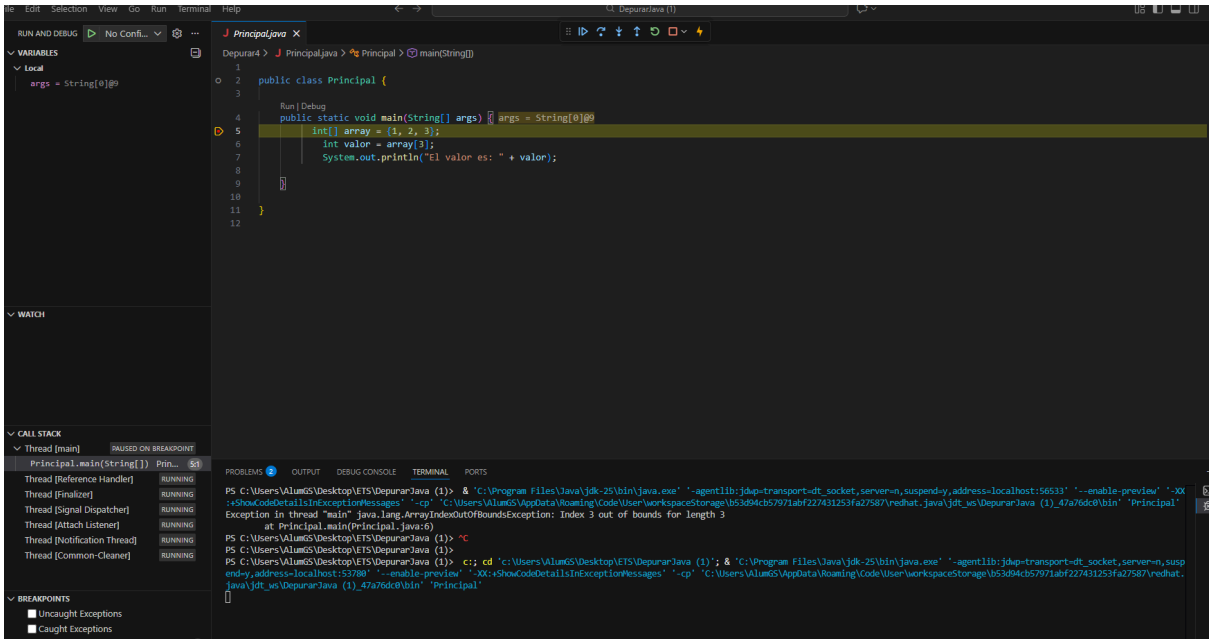
- **¿Qué has tenido que instalar para poder ejecutarlo?**

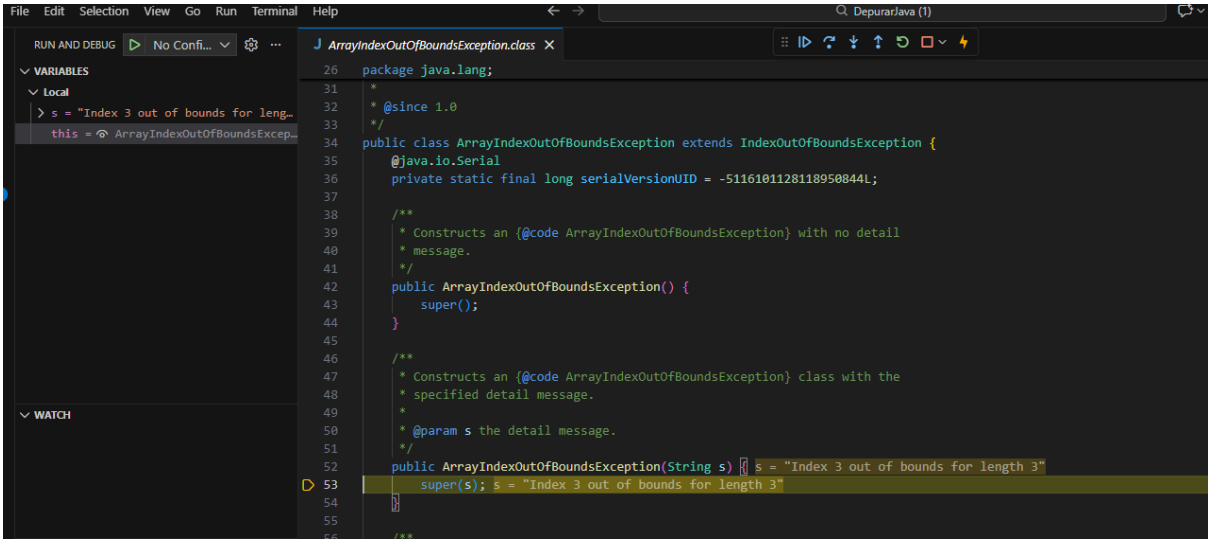
Puede ejecutarlo pero me pide que instale la extension de Java en vscode.

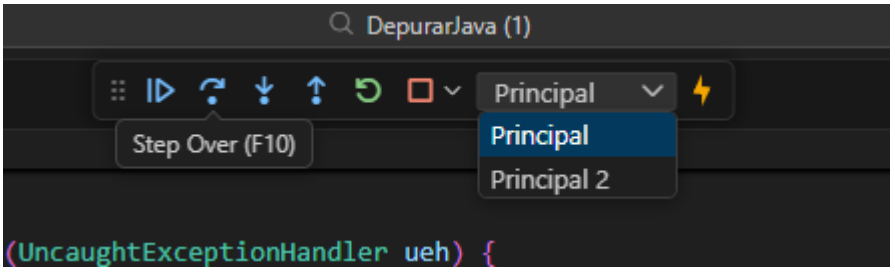


Pero aunque descarguemos la extensión aún no nos deja ejecutarlo. Tenemos que descargarnos la aplicación jdk de java.

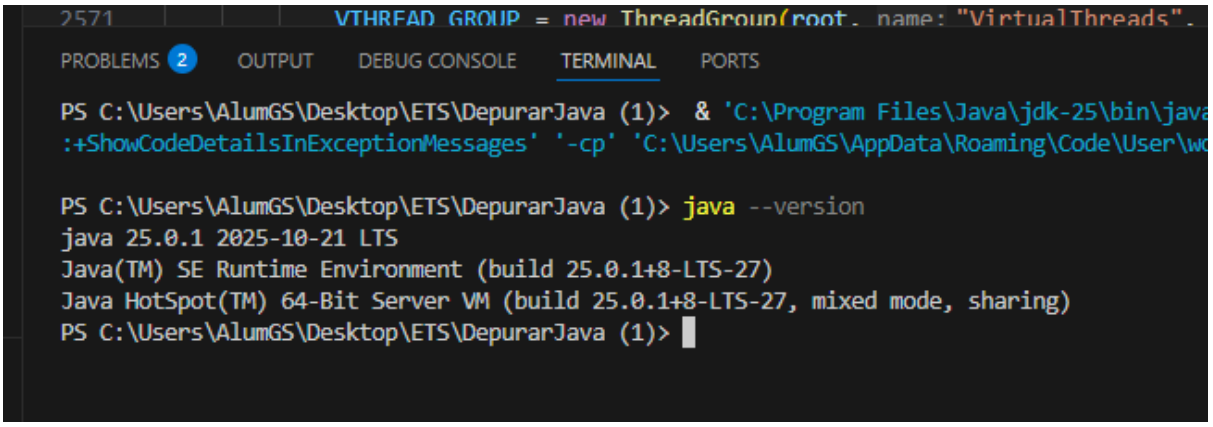
Pon puntos de ruptura y ejecuta el código paso a paso como hiciste en la Actividad 2.
Coloco los breakpoint en las líneas 2 y 5. Y podemos ver como al entrar dentro del bucle, cada linea nos envia a una zona del codigo distinto de otro documento llamado “ArrayIndexOut ofBoundsException”(captura 2). Podemos ir de una zona a otra con el relampago donde pasamos de principal 1 a principal 2 (captura 3).







Muestra lo que te devuelve y coméntalo.
Ya hice el comando de java, y me devuelve esto:



- ¿Qué hace cada uno de los elemento de Java que tienes instalados?
  - Java 25.0.1 LTS: es la versión de java que tenemos instalada.
  - Java SE Runtime Environment (JRE): es el entorno de ejecución. Este elemento es el conjunto de librerías y herramientas básicas que permiten ejecutar programas Java.
  - Java HotSpot 64-Bit Server VM (JVM): es la máquina virtual de Java, es decir, el que realmente ejecuta este programa.
- ¿Te cuadra con lo que hemos visto en teoría sobre los lenguajes compilados?

Por eso Java se considera un lenguaje compilado, pero no se compila directamente a código máquina como C++, sino a un código intermedio que luego ejecuta la máquina

virtual. Por eso decimos que Java es multiplataforma y es como hemos aprendido de lenguajes compilados.

Actividad 6: Compilando y generando el Bytecode de la solución

En este caso para hacer proceso tendremos que ejecutar el siguiente comando en la terminal:

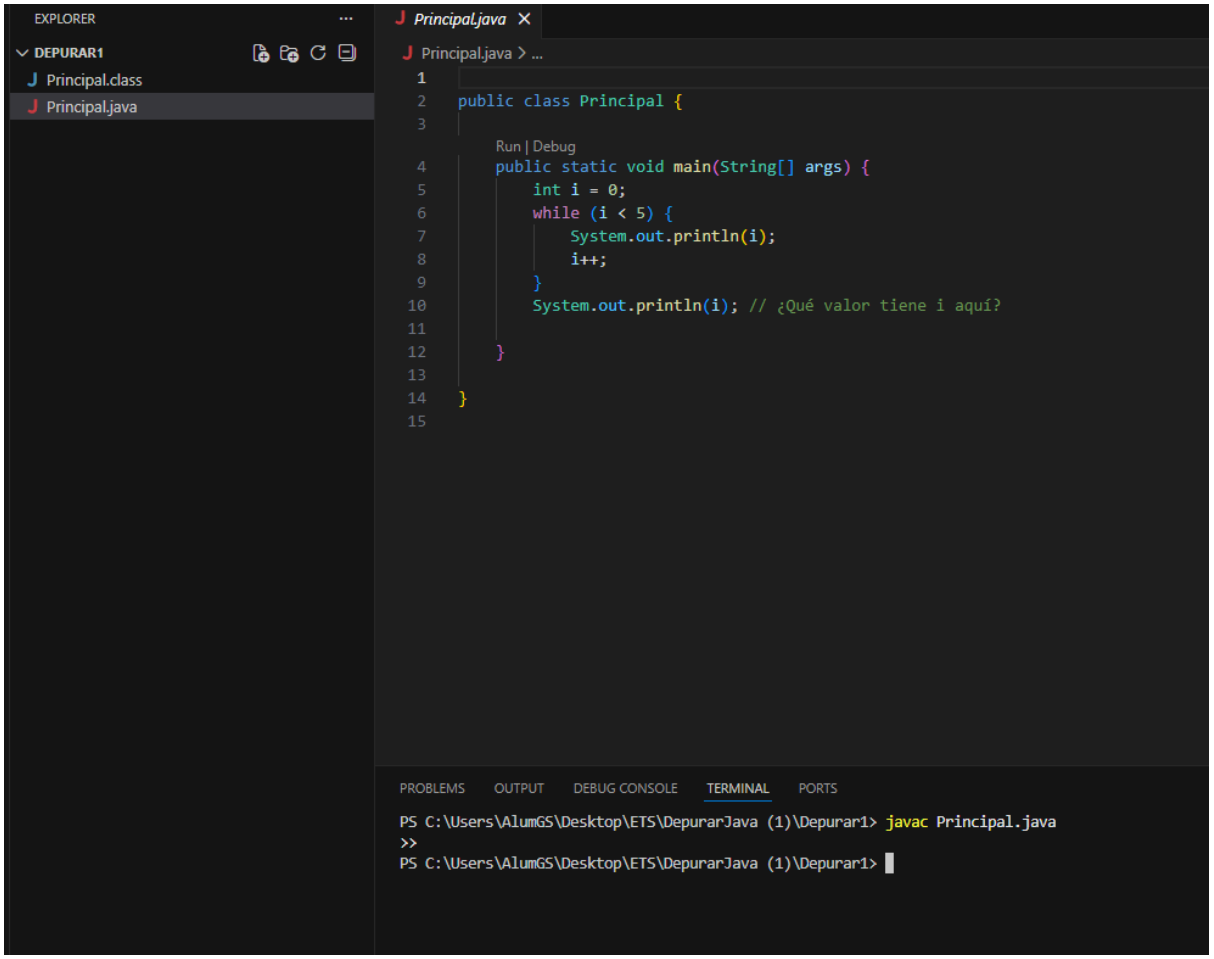
- **javac Principal.java**

Ejecuta el código y comenta que ha pasado

Lo he ejecutado, y me ha sucedido un error:

```
use --help for a list of possible options
PS C:\Users\AlumGS\Desktop\ETS\DepurarJava (1)> javac Principal.java
error: file not found: Principal.java
Usage: javac <options> <source files>
use --help for a list of possible options
PS C:\Users\AlumGS\Desktop\ETS\DepurarJava (1)>
```

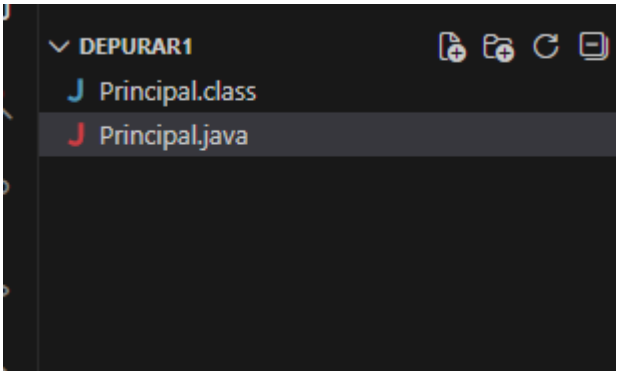
Encontre el error anterior, y es que estaba en una carpeta con muchas carpetas a su vez, pero al volver abrir el vscode pero desde el depurar1 ya me funciono:



Lo que me ha sucedido es que se me ha creado un nuevo archivo .class con el codigo compilado.

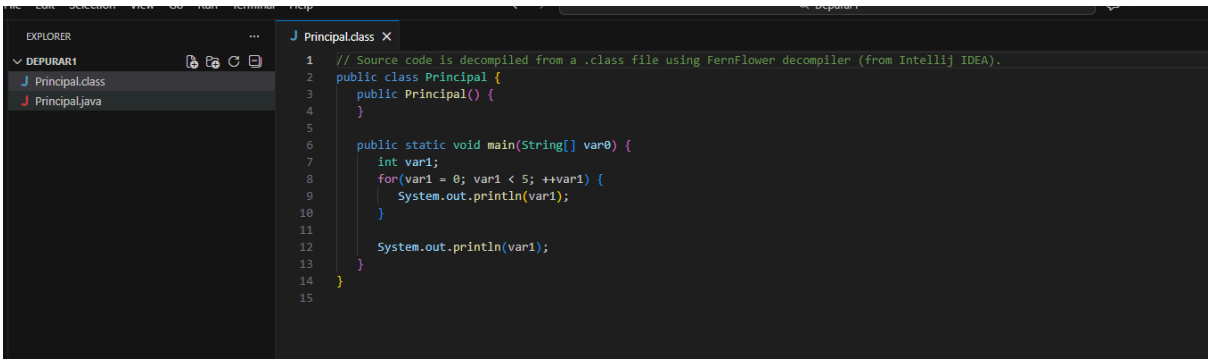
- **¿Se te ha generado el Bytecode?**

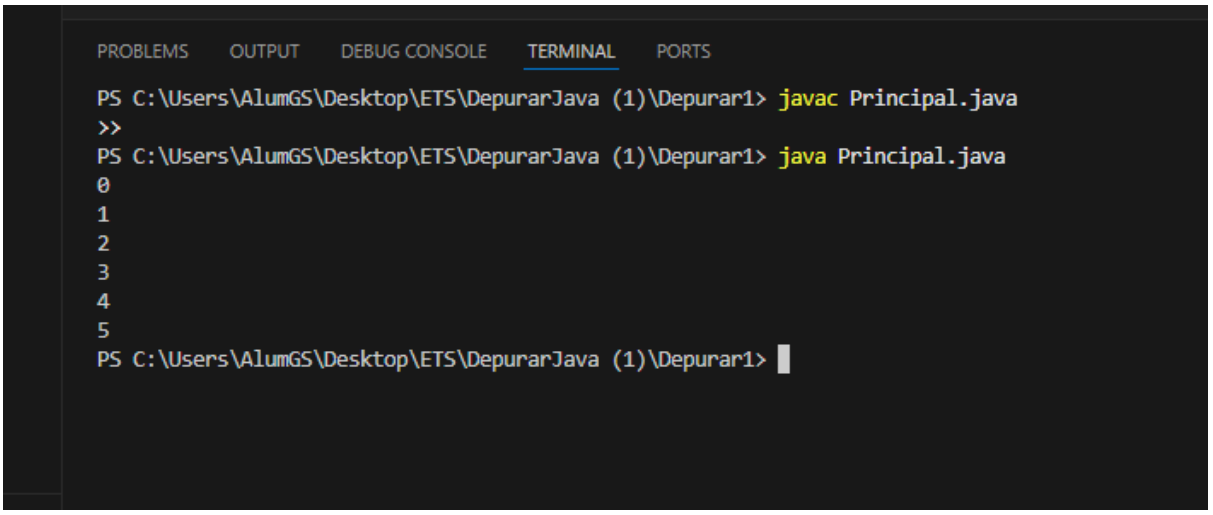
Sí, un archivo con código compilado .class:



- **¿Puedes abrir el fichero con un editor de texto?**

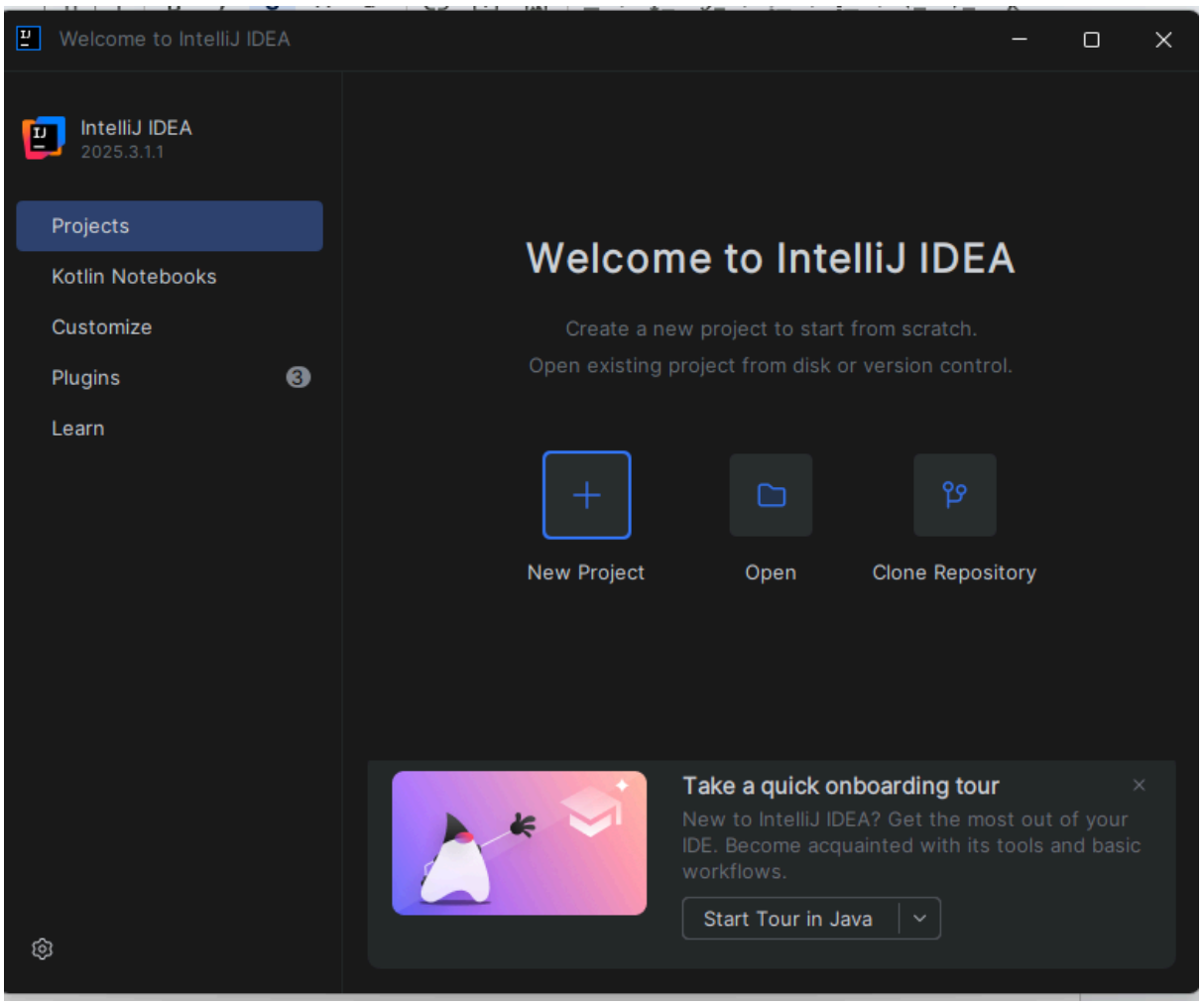
Sí, se puede abrir el fichero .class con un editor de texto, pero porque el editor solo muestra una versión descompilada del .class pero no es el original.



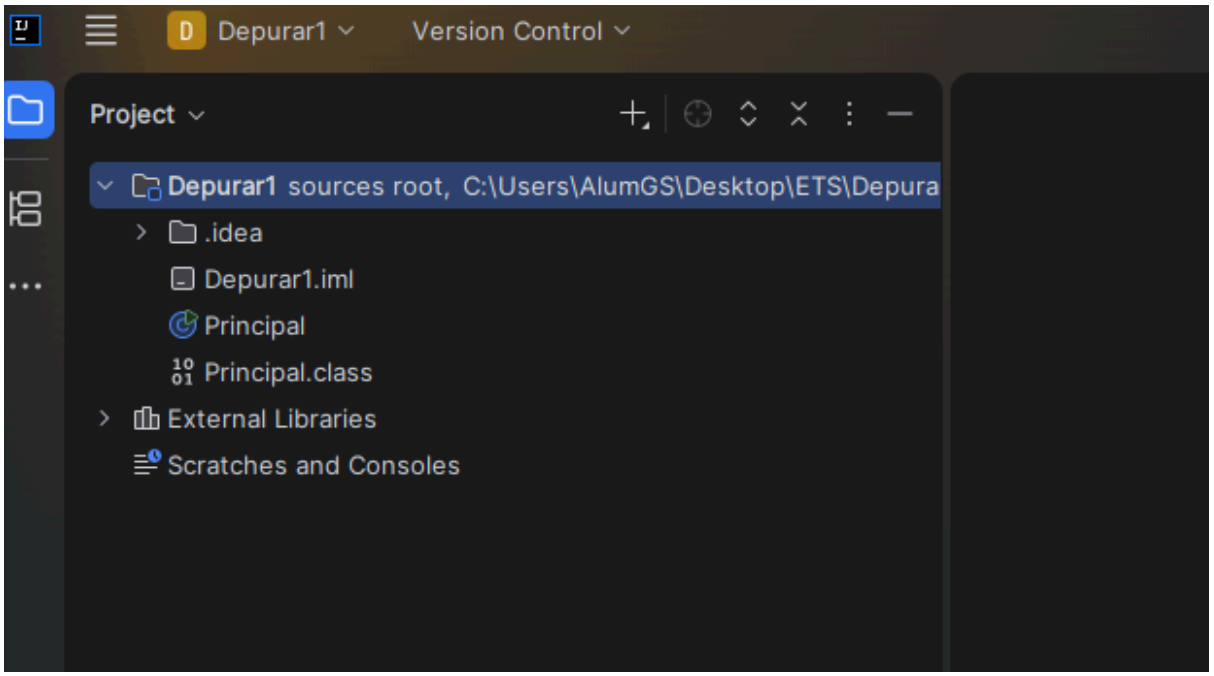


Actividad 6: IDE IntelliJ

- **Descarga e instala el IDE IntelliJ. https://www.jetbrains.com/es-es/idea/)**

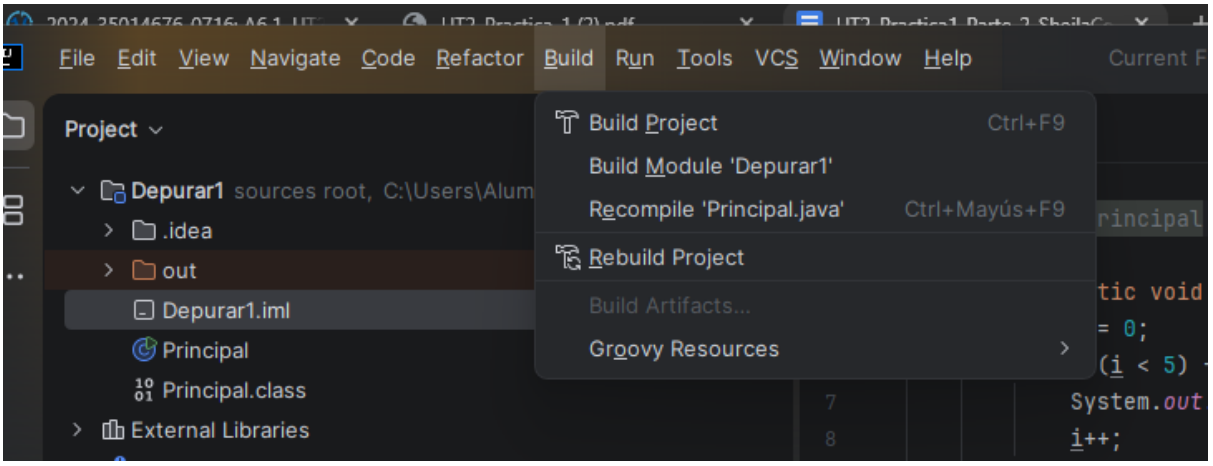


Una vez hecho esto abre la carpeta Depurar1 y repite la actividad 5 pero con este IDE (la parte de ejecutar el comando java –version no hace falta).

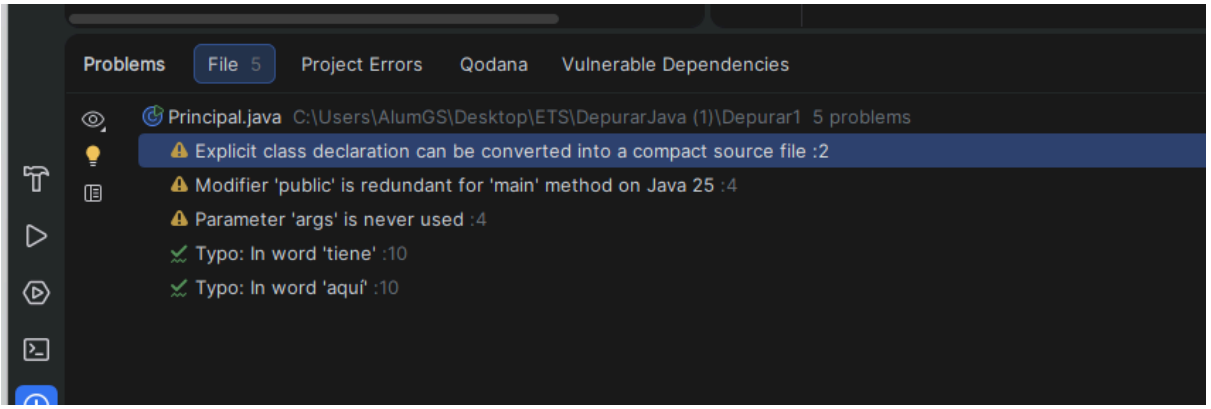


- **¿Has podido encontrar las mismas opciones de depuración que tenías para el visual Studio Code?**

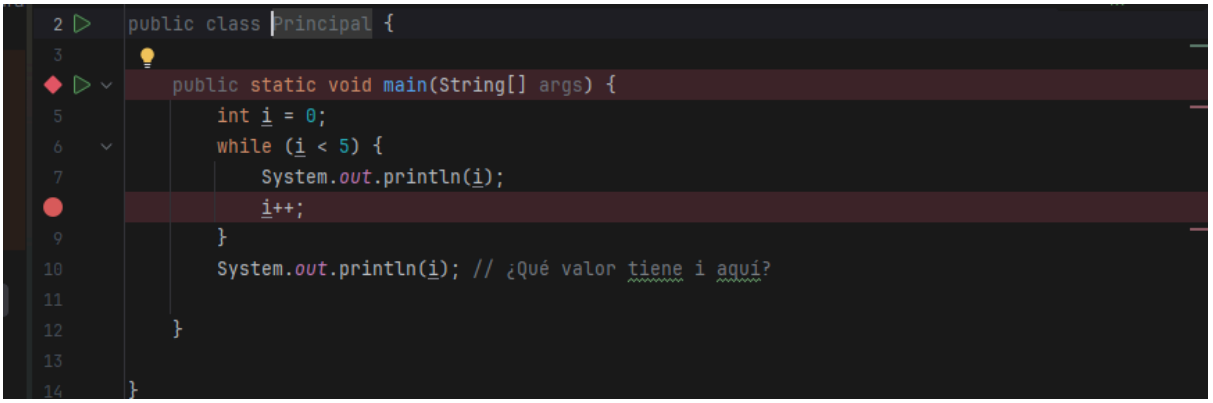
No, esta vez en de de usar new Terminal, tenemos la opción de “Build”. Lo que nos compila automáticamente, a diferencia de la terminal en vscode.



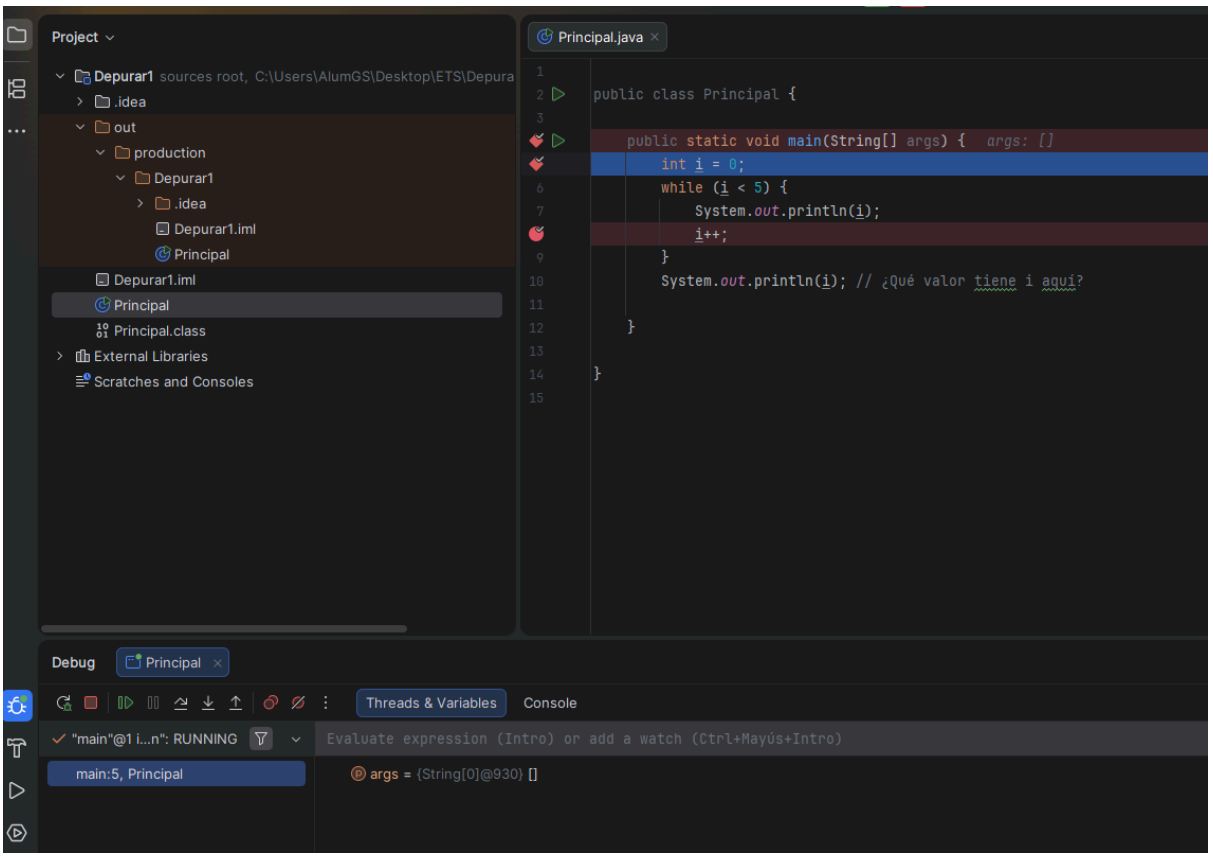
También ahora podemos ver los errores que tiene el código:



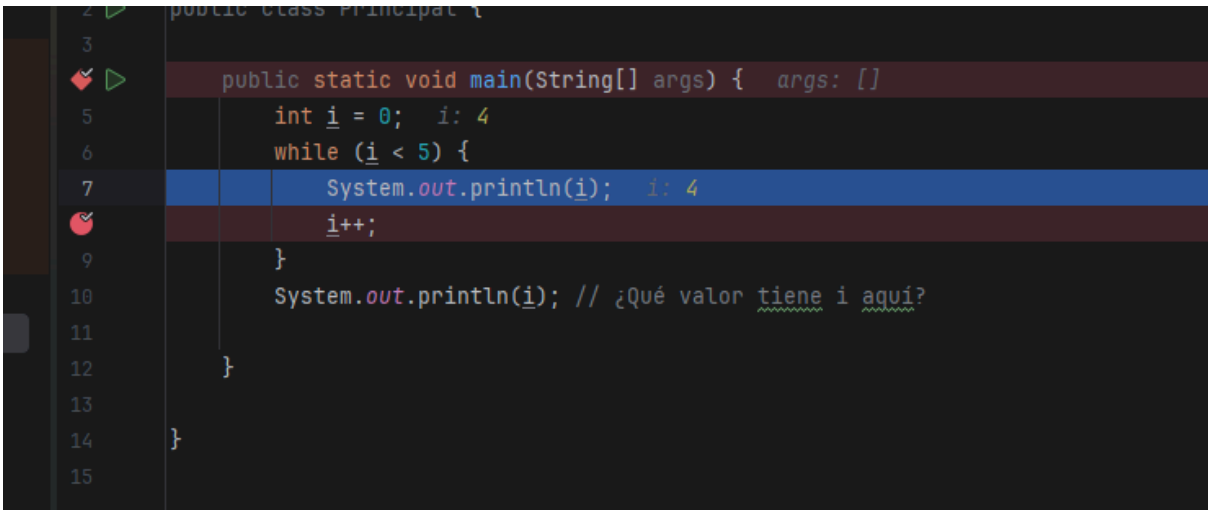
Coloco 2 breakpoint para ver qué pasa si uso el debug ahora:



Aquí en IDEA los botones del debug, están abajo en la izquierda, donde está el build ejecutado.

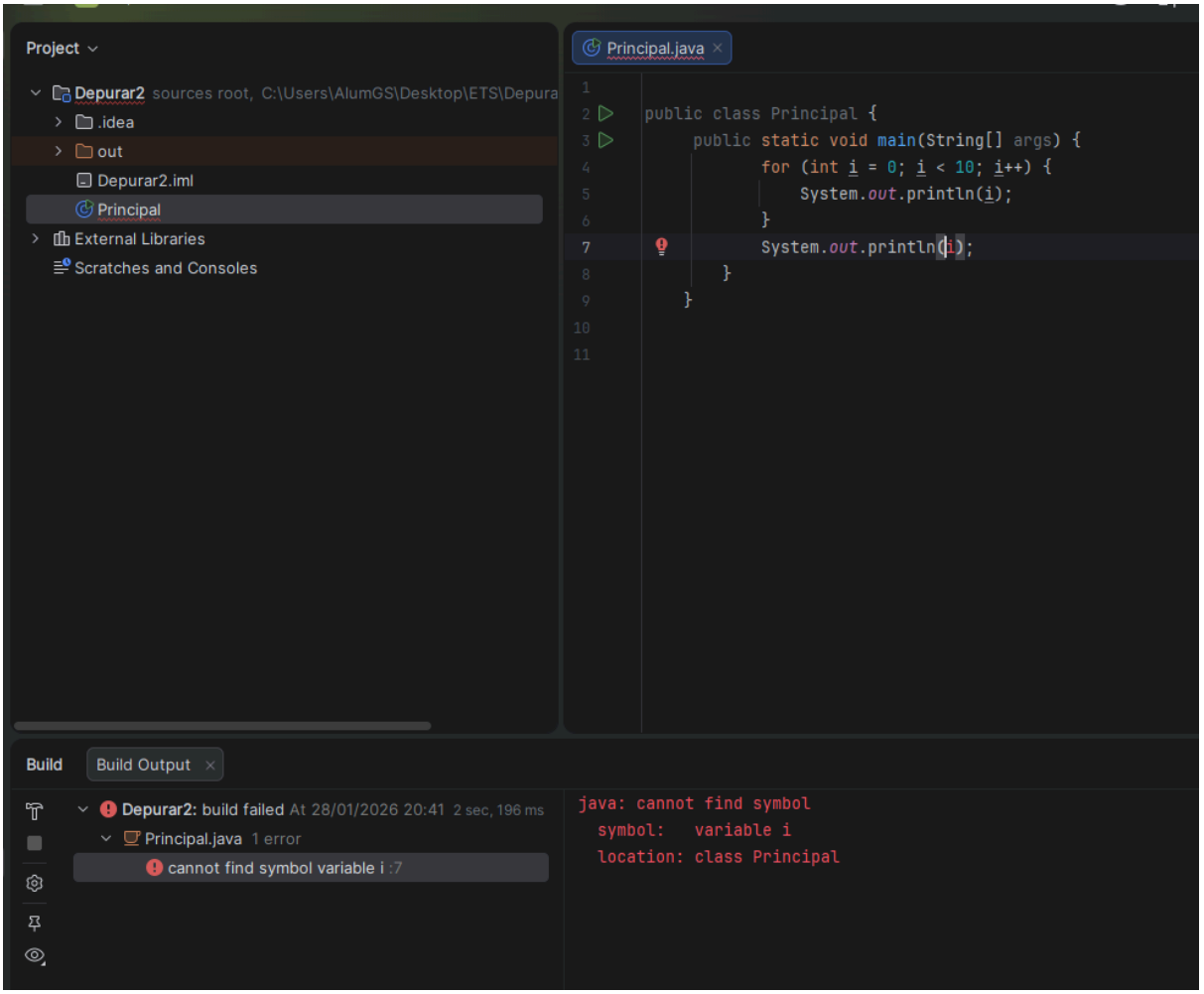


Ahora podemos usar el debug, y vemos como se ejecuta cada línea hasta llegar el bucle a 5.



Actividad 7:

Abre desde IntelliJ la carpeta “Depurar2” y ejecuta el fichero .java.

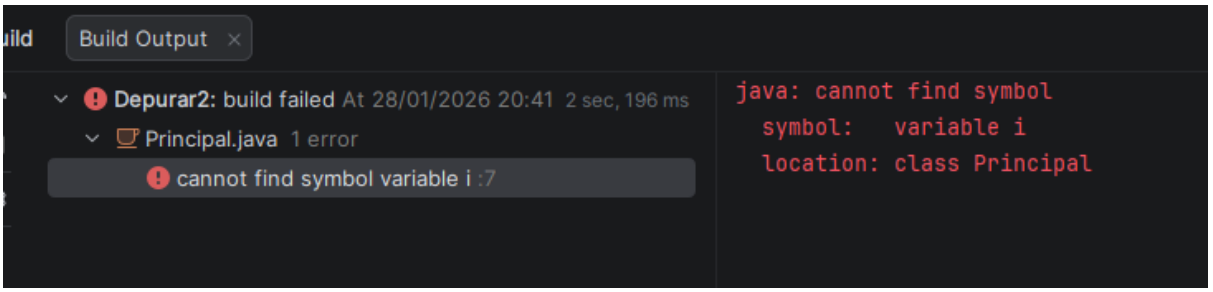


- **¿Puedes ejecutarlo?**

Si puede ejecutarlo pero me salta que hay una variable errónea en el código.

- **¿Te devuelve algún error?**

Sí, una variable (i) que no existe y me lo señala:



- **¿A que se debe (pista: comenta la línea que te da el problema y depura paso a paso observando los valores de las variables)?**

Lo que sucede es que el “i” solo esta nombrada dentro del for pero en el print como no la hemos llamado fuera de la funcion, para print es como si no existiera, por eso debemos llamar a la variable “i” fuera del bucle. Además debemos quitarle el int a la variable i porque ya la nombramos al principio, y ya vale para todo el codigo.

```java
public class Principal {
    public static void main(String[] args) {
        int i;
        for (int i = 0; i < 10; i++) {
            System.out.println(i);
        }
        System.out.println(i);
    }
}
```

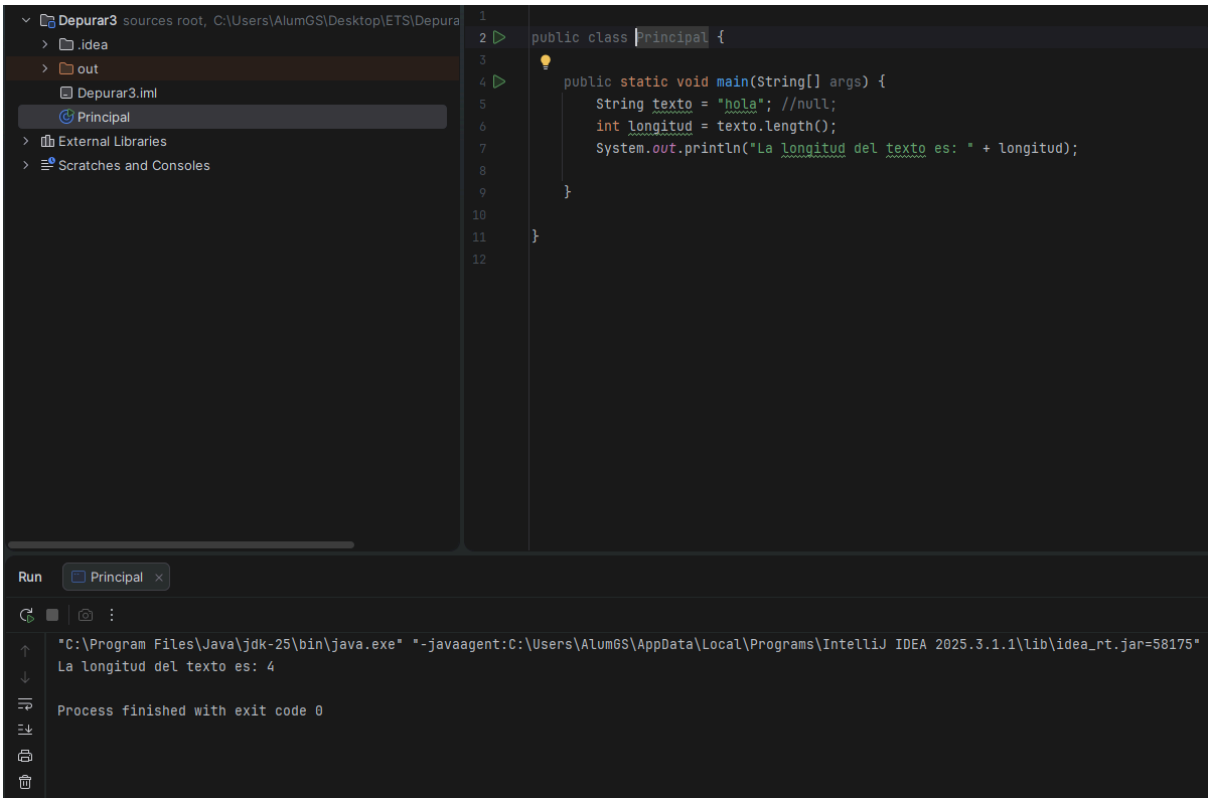
```java
public class Principal {
    public static void main(String[] args) {
        int i;
        for (i = 0; i < 10; i++) {
            System.out.println(i);
        }
        System.out.println(i);
    }
}
```

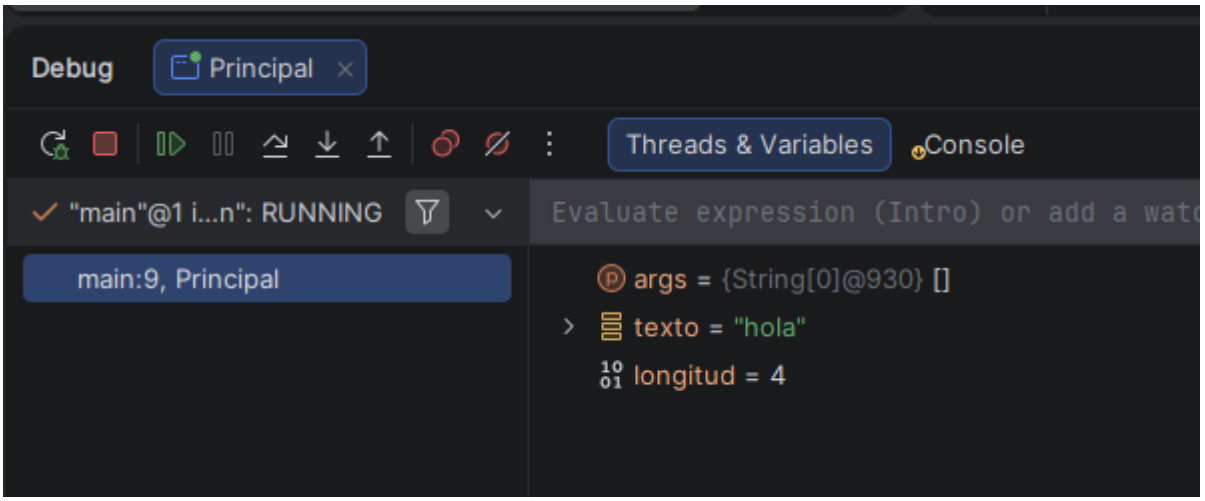
Y ya no hay ningún error:

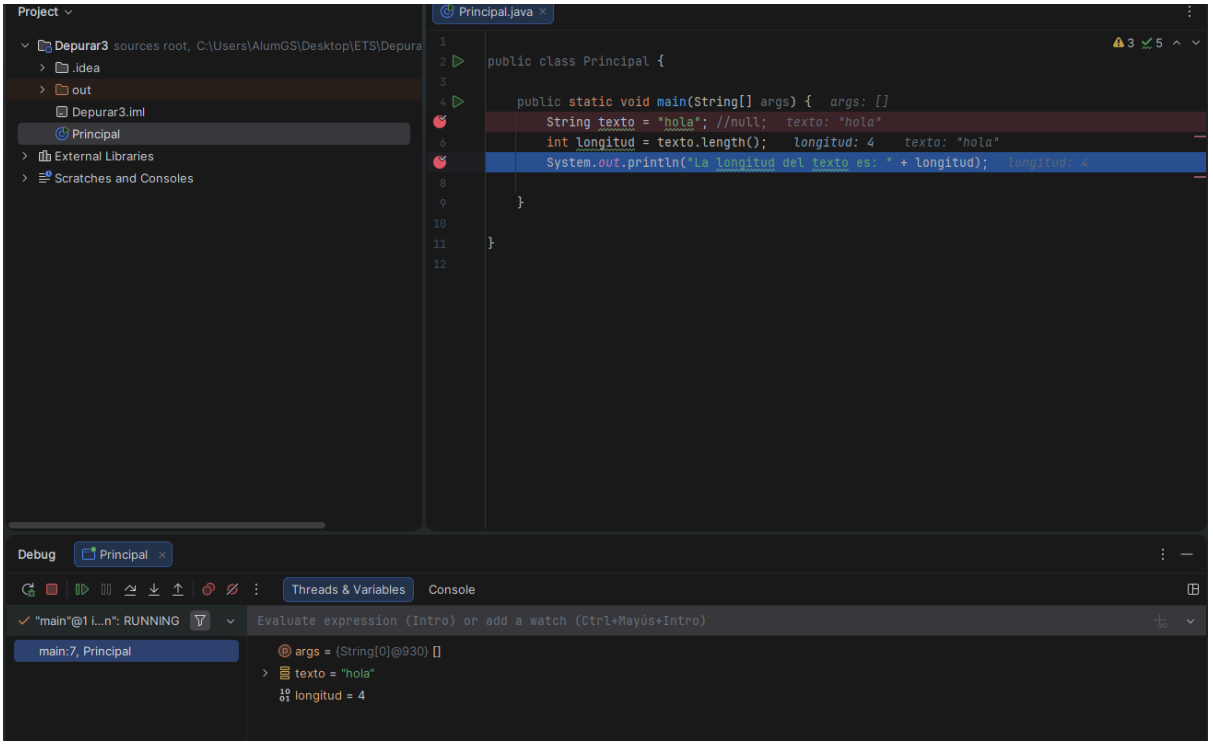
```
3
4
5
6
7
8
9
10

Process finished with exit code 0
```

- **Haz lo mismo con la carpeta “Depurar3”**

Hacemos lo mismo con este archivo pero esta vez, no nos sale ningún error en el código al ejecutarlo, por lo que decidí usar el debug para ver cada línea pero todas van perfectamente, no hice ningún cambio.



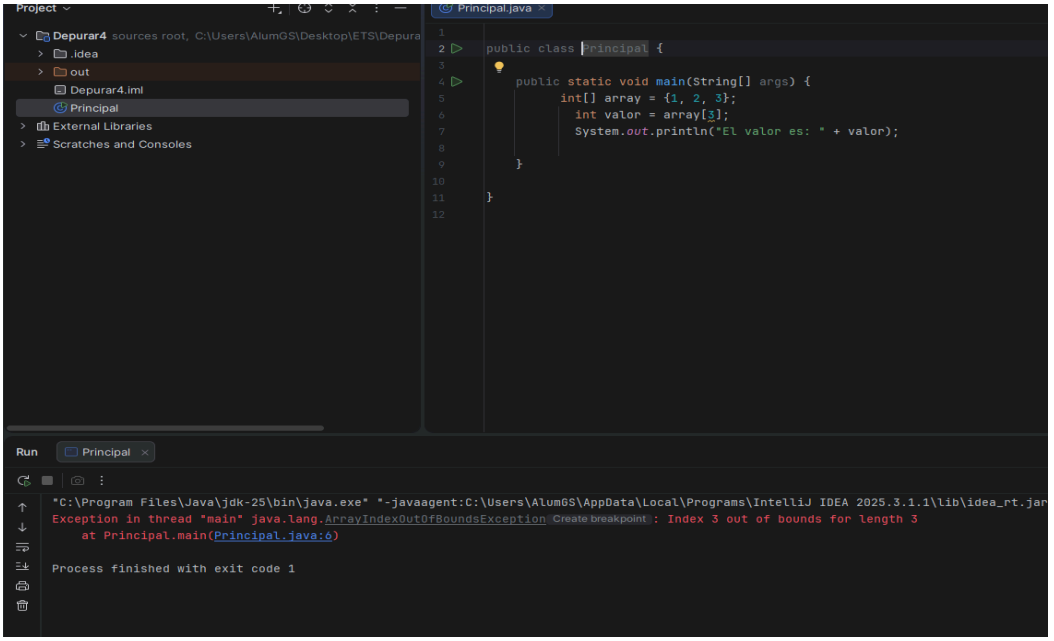




Actividad 8:

Abre desde IntelliJ la carpeta “Depurar4” y ejecuta el fichero .java. Comenta porqué se produce el error y modifica el código para que se muestre algo por pantalla.

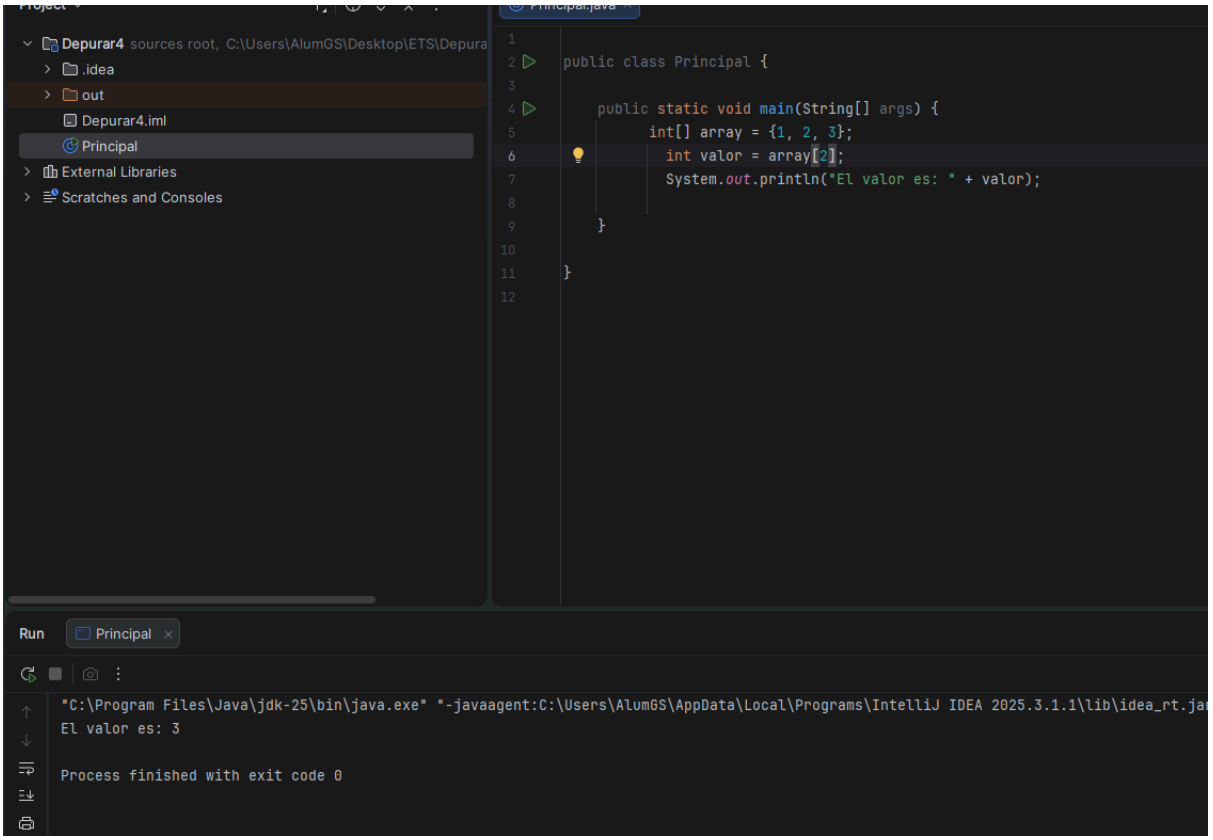
Ejecuto el fichero y me sale el error siguiente:



El error nos está diciendo que el código está intentando acceder a una posición de un array que no existe. Esto es porque pides que la array sea de 3 pero las arrays empiezan a contar desde 0, por lo que estás pidiendo 4 números de la array.
Solo cambie en la linea 6, el 3 por el 2, y ya debería funcionar:

```
2  public class Principal {
3
4      public static void main(String[] args) {
5          int[] array = {1, 2, 3};
6          int valor = array[2];
7          System.out.println("El valor es: " + valor);
8
9      }
10
11 }
```

Y ejecutamos para comprobar y funciona:

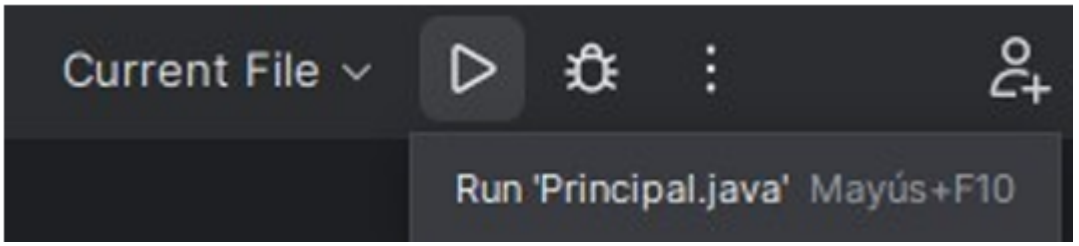


Porque lo vi a ojo pero si hacemos un debug, te sale el tamaño de la array que estas pidiendo:

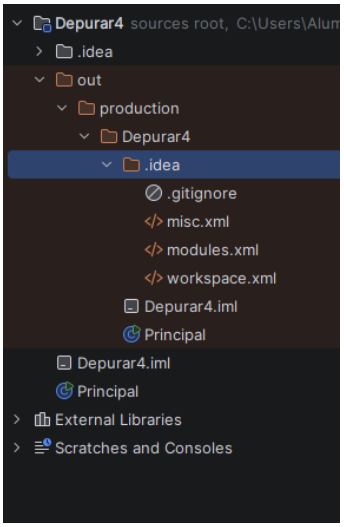
```
2  public class Principal {
3
4      public static void main(String[] args) {   args: []
5          int[] array = {1, 2, 3};   array: [1, 2, 3]
6          int valor = array[2];   array: [1, 2, 3]
7          System.out.println("El valor es: " + valor);
8
9      }
10
11 }
```

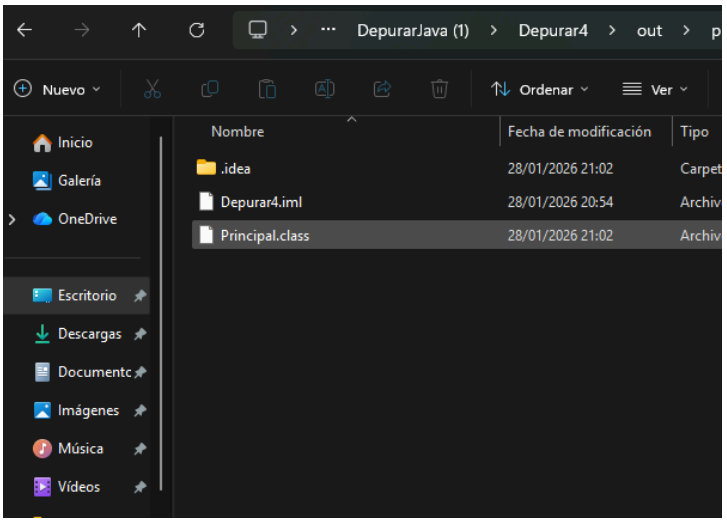
Actividad 9:

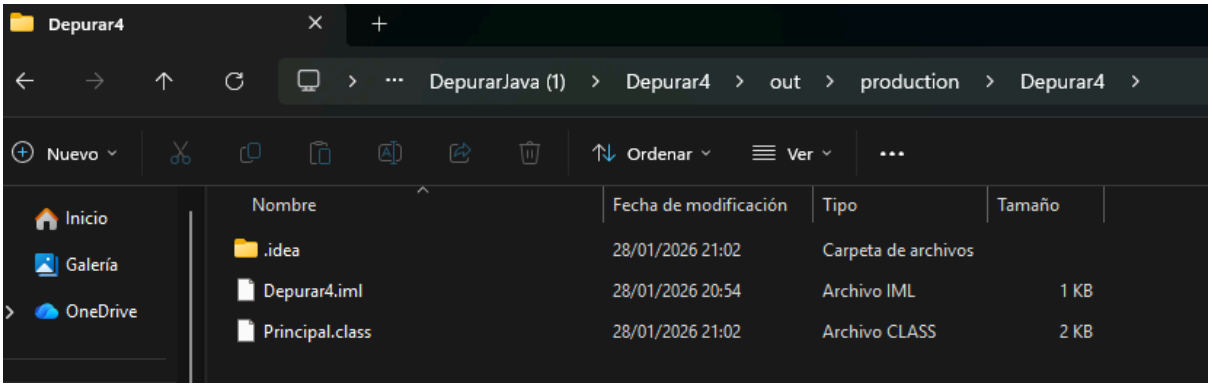
Con el código corregido vamos ahora a hacer el "Build" de la solución. Para ello nos vamos a las opciones del IDE en la parte superior: "Build"→ "Build Project" (o CTRL+F9) Si ejecutamos la aplicación en modo normal:



Lo acabo de hacer y me aparece un mensaje de que el código ha sido "reloaded". Se me creó el archivo .class pero dentro de la carpeta out y en oculto. Tuve que ir hasta la carpeta directamente fuera de IDEA porque esta aplicación de por si no te muestra el archivo .class.





¿Se te ha generado el Bytecode?
Sí, pero el IDEA no te muestra como tal el archivo .class, debes salir de la aplicación para encontrarlo dónde está la carpeta del código.
¿Dónde se te ha generado?

Y estaba en la ruta de DepurarJava/ Depurar4/out/production/Depurar4 y ahí veremos nuestros dos archivos: